

zØmbie zØne

Parc d'attractions fictif post-apocalyptique

DOSSIER DE PROJET

Titre Professionnel — Concepteur Développeur d'Applications — Niveau 6
RNCP37873 — O'clock

Candidat : Stéphane Rochard
Date de soutenance : 9 juin 2026

GitHub : <https://github.com/5h4r0/z0mbiez0ne>

Production : <https://sharo.fr> | API docs : <https://sharo.fr/api/docs>

Table des matières

- Table des matières..... 2
- 1. Compétences du référentiel couvertes..... 5
 - Activité Type 1 — Développer une application sécurisée 5
 - Activité Type 2 — Concevoir et développer une application sécurisée organisée en couches..... 5
 - Activité Type 3 — Préparer le déploiement d'une application sécurisée 5
- 2. Cahier des charges et expression des besoins 7
 - 2.1 Contexte 7
 - 2.2 Cibles et contraintes ergonomiques..... 7
 - 2.3 User stories — Visiteur / Membre 7
 - 2.4 User stories — Administrateur 7
 - 2.5 Contraintes techniques 8
- 3. Présentation du candidat et contexte..... 9
 - 3.1 Parcours..... 9
 - 3.2 Contexte du projet..... 9
- 4. Gestion de projet 10
 - 4.1 Méthodologie SCRUM..... 10
 - 4.2 Outil de gestion — GitHub Projects..... 10
 - 4.3 Traçabilité issue → commit → merge (exemple réel) 10
 - 4.4 Conventions de code 11
 - 4.5 Planning des sprints..... 11
- 5. Spécifications fonctionnelles..... 12
 - 5.1 Architecture logicielle 12
 - Rôle de chaque couche et stratégie de sécurité..... 12
 - 5.2 Maquettes et enchaînement des écrans..... 13
 - Description des écrans principaux 19
 - 5.3 Modèle Conceptuel de Données (MCD) 20
 - Cardinalités et justifications 20
 - Choix de conception notables 21
 - 5.4 Cas d'utilisation — Diagramme..... 21
 - 5.5 Diagramme de séquence — Cycle d'authentification JWT 22
- 6. Spécifications techniques et sécurité..... 26
 - 6.1 Stack technique justifiée 26
 - 6.2 Sécurité — mesures implémentées..... 26
 - JWT et stockage des tokens..... 26
 - Référence OWASP Top 10 — couverture projet..... 27
 - 6.3 Traçabilité d'une fonctionnalité dans le code (CP6)..... 27
- 7. Réalisations — Extraits de code significatifs 28
 - 7.1 Authentification JWT — Backend..... 28

Controller loginUser	28
apiFetch — Intercepteur 401 avec déduplication.....	28
7.2 Composants d'accès aux données — Prisma (CP8).....	28
Pagination — helper réutilisable.....	28
Transaction Prisma — createOrderLine avec vérification capacité	29
7.3 Interface utilisateur — React.....	29
Guard bfcache — DashboardPage	29
8. Éléments de sécurité de l'application	30
8.1 Veille sécurité effectuée.....	30
8.2 Failles identifiées et corrections	30
9. Plan de tests	31
9.1 Stratégie de tests.....	31
9.2 Tests d'intégration — 116 tests	31
9.3 Tests unitaires — 35 tests	31
9.4 Infrastructure de tests	31
10. Jeu d'essai — Fonctionnalité la plus représentative.....	33
10.1 Scénarios testés.....	33
Scénario 1 — Login avec identifiants valides.....	33
Scénario 2 — Login avec mauvais mot de passe.....	33
Scénario 3 — Token expiré → Refresh automatique	33
Scénario 4 — Accès à une ressource sans authentification	33
Scénario 5 — Logout + tentative d'utilisation du refreshToken révoqué.....	34
10.2 Analyse des écarts	34
11. Déploiement — Procédure et documentation.....	35
11.1 Infrastructure de production	35
11.2 Architecture Docker (multi-stage).....	36
11.3 Pipeline CI/CD — GitHub Actions.....	37
11.4 Procédure de mise à jour	38
12. Difficultés rencontrées et améliorations	39
12.1 Difficultés techniques.....	39
Race conditions sur le refresh token (React StrictMode)	39
bfcache — restauration de page sans re-exécution JavaScript.....	39
Docker multi-stage et migrations Prisma.....	39
Biome 2.x en monorepo.....	39
12.2 Améliorations futures	39
12.3 Conclusion	40
Annexes	41
A. Endpoints API complets	41
B. Variables d'environnement.....	41

1. Compétences du référentiel couvertes

Le projet zØmbie zØne couvre l'ensemble des compétences professionnelles du référentiel CDA (REAC TP-01281, millésime 04, juillet 2024). Le tableau ci-dessous précise, pour chaque compétence, le niveau atteint et les éléments du projet qui en témoignent.

Activité Type 1 — Développer une application sécurisée

Compétence	Statut	Mise en œuvre
CP1 — Installer et configurer l'environnement	☐ Couverte	Monorepo npm workspaces, Docker Compose dev/prod, PostgreSQL, Biome 2.x, GitHub Actions CI/CD, variables d'environnement séparées par env
CP2 — Développer des interfaces utilisateur	☐ Obligatoire	SPA React 19 + Vite + React Router 7, composants typés TypeScript, responsive mobile-first, accessibilité RGAA 60%
CP3 — Développer des composants métier	☐ Obligatoire	API REST Express 5 TypeScript, controllers, middlewares requireAuth/requireRole, validation Zod, gestion erreurs custom
CP4 — Gestion de projet informatique	☐ Obligatoire	GitHub Projects (Kanban), sprints hebdo, commits conventionnels EN, lien issues ↔ commits ↔ PRs, Biome + conventions nommage

Activité Type 2 — Concevoir et développer une application sécurisée organisée en couches

Compétence	Statut	Mise en œuvre
CP5 — Analyser les besoins et maquetter	☐ Obligatoire	Cahier des charges, user stories, maquettes Figma mobile-first, enchaînement écrans formalisé (voir diagramme section 5), arborescence du site
CP6 — Définir l'architecture logicielle	☐ Obligatoire	Architecture multicouche SPA ↔ API REST ↔ BDD, rôle de chaque couche documenté, fonctionnalité traçable route → controller → Prisma → BDD
CP7 — Concevoir et mettre en place une BDD	☐ Obligatoire	PostgreSQL 16, MCD/MPD (voir section 5), Prisma migrations versionnées, règles nommage snake_case, intégrité FK, BDD test isolée restaurable
CP8 — Composants d'accès aux données SQL	☐ Obligatoire	Prisma ORM : CRUD, transactions \$transaction, select explicite, cas d'exception gérés, confidentialité (password_hash exclu des réponses)

Activité Type 3 — Préparer le déploiement d'une application sécurisée

Compétence	Statut	Mise en œuvre
------------	--------	---------------

CP9 — Préparer et exécuter les plans de tests	☐ Obligatoire	Vitest + Supertest : 116 tests intégration + 35 tests unitaires = 151 tests. Plan documenté dans TESTS.md. Environnement de test isolé.
CP10 — Préparer et documenter le déploiement	☐ Couverte	DEPLOY.md complet (procédure VPS, Docker multi-stage, migrations, secrets, checklist, pièges connus)
CP11 — Contribuer à la mise en production DevOps	☐ Couverte	GitHub Actions : lint → test → build → deploy SSH. Docker Compose prod. Nginx reverse proxy + SSL Let's Encrypt.

2. Cahier des charges et expression des besoins

2.1 Contexte

zØmbie zØne est un parc d'attractions fictif à thème post-apocalyptique. L'objectif est de concevoir et développer l'application web complète : vitrine immersive, réservation de billets par sessions d'activités, espace client et backoffice d'administration. Ce projet est réalisé dans le cadre de la formation O'clock CDA, simulant un contexte professionnel réel.

2.2 Cibles et contraintes ergonomiques

- Cible principale : adolescents et jeunes adultes (16-30 ans)
- Design immersif et moderne, adapté au thème horrifique
- Mobile-first — utilisation mobile importante
- Accessibilité RGAA 2.1 à 60% minimum

2.3 User stories — Visiteur / Membre

Fonctionnalité	Acteur	Description
Consulter les activités	Visiteur	Lister toutes les activités, filtrer par catégorie, voir le détail (description, prix, sessions)
Créer un compte	Visiteur	S'inscrire avec email/mot de passe, validation de la politique de sécurité
S'authentifier	Visiteur	Se connecter, se déconnecter, réinitialiser son mot de passe
Gérer son panier	Membre	Ajouter/modifier/supprimer des sessions avec quantité, voir le total HT + TTC
Passer commande	Membre	Confirmer ou annuler sa commande (statut Pending → Confirmed/Cancelled)
Historique commandes	Membre	Consulter ses commandes passées et en cours, voir le détail de chaque commande
Gérer son compte	Membre	Modifier son profil, changer son mot de passe, supprimer son compte (soft delete RGPD)

2.4 User stories — Administrateur

Fonctionnalité	Acteur	Description
CRUD Activités	Admin	Créer, lire, modifier, supprimer une activité + upload image bannière et miniature WebP
CRUD Sessions	Admin	Créer, lire, modifier, supprimer une session — blocage si des commandes existent
CRUD Catégories	Admin	Créer, lire, modifier, supprimer une catégorie — blocage si liée à une activité
Gestion Utilisateurs	Admin	Lire et modifier les comptes, soft delete (jamais de suppression physique)

Gestion Commandes	Admin	Lire toutes les commandes, modifier le statut (Confirmed, Refunded...)
-------------------	-------	--

2.5 Contraintes techniques

Contrainte	Exigence
Authentification	JWT access token (15 min) + refresh token (7 j), cookies httpOnly exclusivement — jamais localStorage
Tests	151 tests automatisés (116 intégration + 35 unitaires), pipeline CI/CD
Responsive	Mobile-first, breakpoints CSS définis dans le design system
Accessibilité	RGAA 2.1 minimum 60% — attributs ARIA, contrastes, navigation clavier
RGPD	Soft delete sur users/orders, mentions légales, politique cookies
Éco-conception	Images WebP, lazy loading, pagination, select Prisma explicite, build Vite optimisé
SEO	Slugs auto-générés, balises meta, structure sémantique HTML5
Sécurité	Validation Zod tous les inputs, argon2id mots de passe, OWASP Top 10
Déploiement	Docker multi-stage, CI/CD GitHub Actions, VPS Ionos documenté

3. Présentation du candidat et contexte

3.1 Parcours

Passionné par le développement web depuis plusieurs années, j'ai intégré la formation CDA à O'clock après une reconversion professionnelle. Cette formation intensive m'a permis de maîtriser le développement fullstack TypeScript, de la conception de bases de données relationnelles à la mise en production dans une démarche DevOps. Avant cette formation, j'avais développé plusieurs projets personnels en autodidacte, ce qui m'a donné une base solide en JavaScript/TypeScript et une appétence pour les architectures propres et sécurisées.

3.2 Contexte du projet

zØmbie zØne a été réalisé en solo sur plusieurs semaines, en simulant un contexte professionnel rigoureux : cahier des charges défini, méthodologie SCRUM, suivi de tâches sur GitHub Projects, commits conventionnels en anglais, et déploiement continu en production.

Rôle endossé	Responsabilités
Product Owner	Définition du backlog, priorisation des user stories, vision produit
Scrum Master	Organisation des sprints, suivi des tâches, rétrospectives
Lead Dev / Architecte	Choix de la stack, architecture multicouche, patterns et conventions
Développeur Back-End	API REST, authentification JWT, base de données, sécurité
Développeur Front-End	SPA React, routing, state management, composants UI
DevOps / QA	Tests, CI/CD, Docker, déploiement VPS, documentation technique

4. Gestion de projet

4.1 Méthodologie SCRUM

Le projet a été conduit selon la méthodologie Scrum adaptée à un développeur solo. Les rituels et artefacts mis en place couvrent les critères d'évaluation du niveau 'Confirmé' (CP4) : planification suivie tout au long du projet, journal de bord et traçabilité complète.

Rituel / Artefact	Mise en œuvre
Product Backlog	Liste de toutes les issues GitHub classées par priorité (labels : feature, bug, test, docs, chore)
Sprint Planning	Définition des objectifs et tâches de chaque sprint (1 semaine) — issues déplacées dans 'In Progress'
Daily Stand-up	Note quotidienne dans le journal de bord : réalisé / blocage / objectif du lendemain
Sprint Review	Démonstration des fonctionnalités livrées — captures d'écran commentées dans les issues fermées
Sprint Retrospective	Analyse des points d'amélioration techniques et de processus — ajout au backlog si action corrective
Definition of Done	Code lint propre (Biome) + tests passants + PR mergée + déployé sur VPS sans erreur CI/CD

4.2 Outil de gestion — GitHub Projects

GitHub Projects est utilisé comme tableau Kanban avec les colonnes : Backlog → To Do → In Progress → Review → Done. Chaque tâche est une issue GitHub liée à une branche de travail et référencée dans le commit de merge.

4.3 Traçabilité issue → commit → merge (exemple réel)

Voici un exemple concret de la traçabilité complète entre une tâche du backlog et sa mise en production, illustrant le niveau 'Confirmé' (CP4) :

Issue GitHub #22 : 'Remove unused deleted_at columns on activities/categories/sessions'
Labels : chore, database | Milestone : Sprint 5 – Hardening
Description : Ces colonnes sont présentes dans le schéma Prisma mais jamais utilisées (hard delete intentionnel). Créer une migration pour les supprimer.

Branche : fix/remove-deleted-at-activities-categories-sessions

Commits sur la branche :
chore : remove deleted_at from activities schema
chore : remove deleted_at from categories schema
chore : remove deleted_at from sessions schema
chore : run migration
20260602014449_remove_deleted_at_activities_categories_sessions
test : update integration tests after schema change

Pull Request #22 : mergée dans master

CI/CD : lint 🟢 | tests 🟢 (116/116) | build 🟢 | deploy 🟢
Issue #22 : automatiquement fermée (closes #22 dans le body de la PR)

4.4 Conventions de code

Les conventions appliquées correspondent au niveau 'Confortable' (CP4) — linters, formatters, conventions de nommage et de commit appliqués :

Convention	Règle
Commits conventionnels	feat/fix/wip/docs/test/refactor/ci/build/chore + emoji — toujours en anglais
Biome 2.x	Linteur + formateur unifié back + front — zéro warning toléré en CI (lint:prod)
TypeScript strict	Pas de 'any', types explicites sur tous les retours de fonctions
Naming conventions	snake_case BDD, camelCase JS/TS, PascalCase composants React, kebab-case routes
Code comments	En anglais exclusivement — JSDoc sur les fonctions publiques
Guard clauses	Retour précoce plutôt qu'imbrication — lisibilité et défensivité
Lookup objects	Plutôt que switch/else if enchaînés pour les mappings de statuts/rôles

4.5 Planning des sprints

Sprint	Thème	Livrables principaux
Sprint 1	Setup & API base	Monorepo, schéma Prisma, migrations, API categories/activities/sessions, seeding @faker-js
Sprint 2	Authentification	Register/login/refresh/logout JWT httpOnly, requireAuth/requireRole middlewares
Sprint 3	Orders & Frontend	Commandes, panier Zustand, espace client React Router 7, apiFetch intercepteur 401
Sprint 4	Admin & Déploiement	Backoffice CRUD admin, Docker multi-stage, CI/CD GitHub Actions, VPS Ionos
Sprint 5	Tests & Hardening	151 tests (Vitest+Supertest), audit sécurité, corrections anomalies, Swagger

5. Spécifications fonctionnelles

5.1 Architecture logicielle

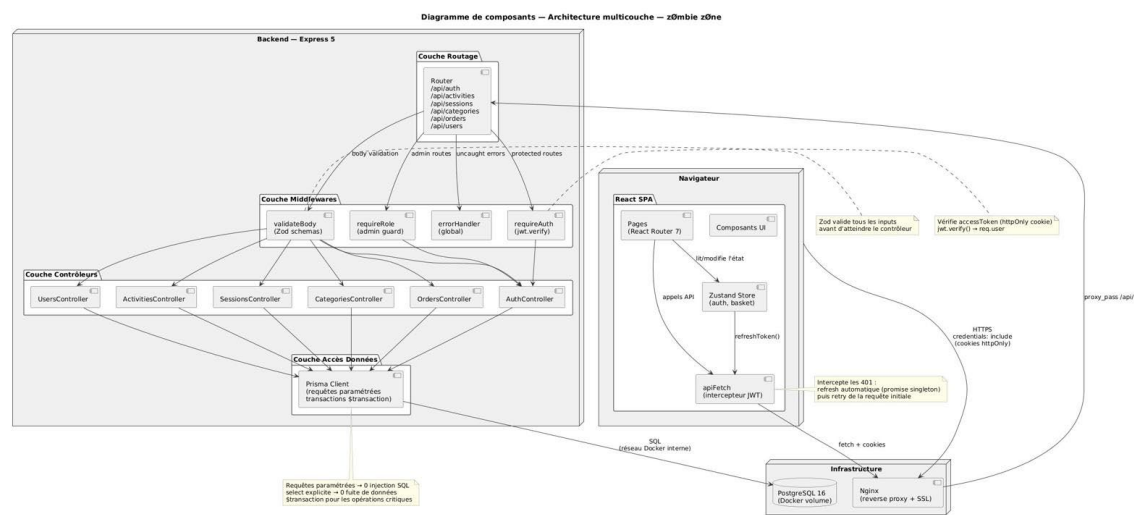
zØmbie zØne adopte une architecture multicouche répartie sécurisée. Les deux applications communiquent exclusivement via l'API REST — le frontend n'a aucun accès direct à la base de données.

COUCHE PRÉSENTATION	React 19 SPA (Vite, React Router 7, Zustand) → fetch + credentials: include (cookies httpOnly)
COUCHE MÉTIER	Express 5 API REST (TypeScript, middlewares) routes → middlewares → controllers → helpers/lib
COUCHE ACCÈS DONNÉES	Prisma ORM (transactions, select explicite) → PostgreSQL 16 (FK, contraintes, soft delete)
COUCHE SÉCURITÉ	JWT httpOnly Zod argon2id CORS strict
INFRASTRUCTURE	Docker Nginx GitHub Actions VPS Ionos

Rôle de chaque couche et stratégie de sécurité

Couche	Responsabilité et sécurité
Présentation (React SPA)	Rendu UI, routing client, state global Zustand. Ne stocke JAMAIS de token (httpOnly). Gère le refresh transparent via apiFetch.
Métier (Express controllers)	Logique applicative, orchestration des accès données, gestion des erreurs typées. Pas de SQL direct — uniquement via Prisma.
Accès données (Prisma)	Requêtes paramétrées (zéro injection SQL), sélection explicite des colonnes (zéro fuite), transactions pour les opérations critiques.
Sécurité (transversale)	Zod valide tous les inputs avant le controller. argon2id hache les mots de passe. JWT en cookies httpOnly+Secure+SameSite=Strict.
Infrastructure	Docker isole les services. Nginx termine SSL et proxifie /api/. GitHub Actions garantit lint+test avant tout déploiement.

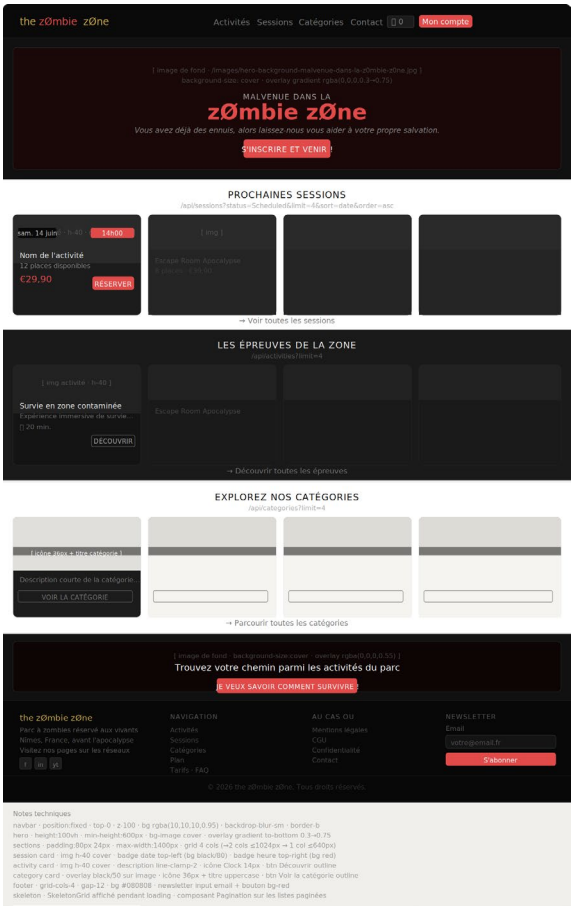
Diagramme de composants

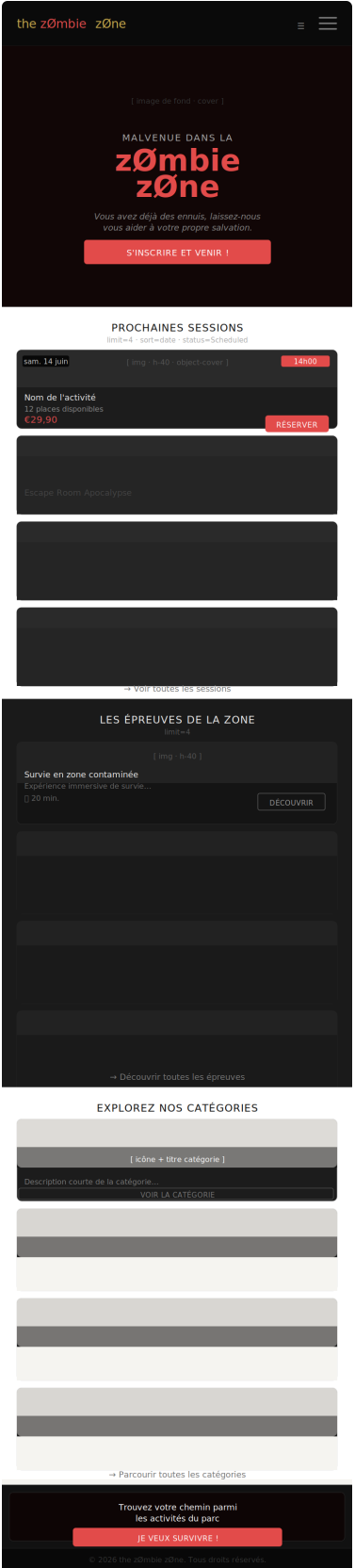


Architecture multicouche — Router → Middleware → Controller → Prisma → DB

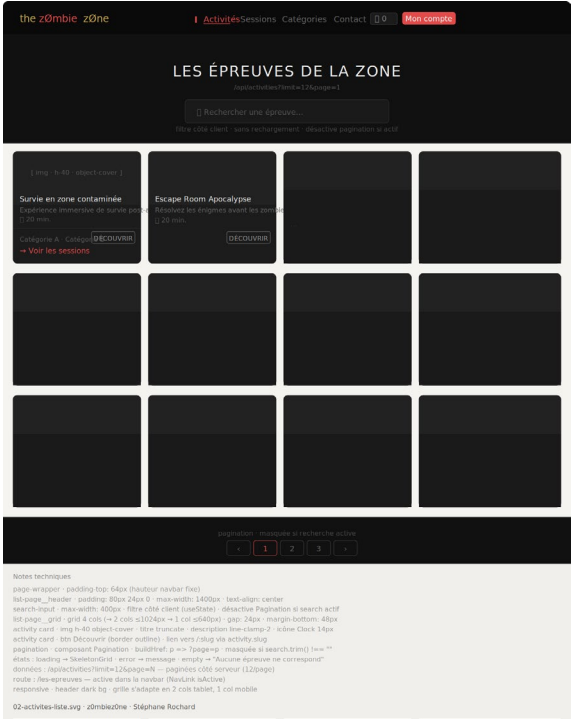
5.2 Maquettes et enchaînement des écrans

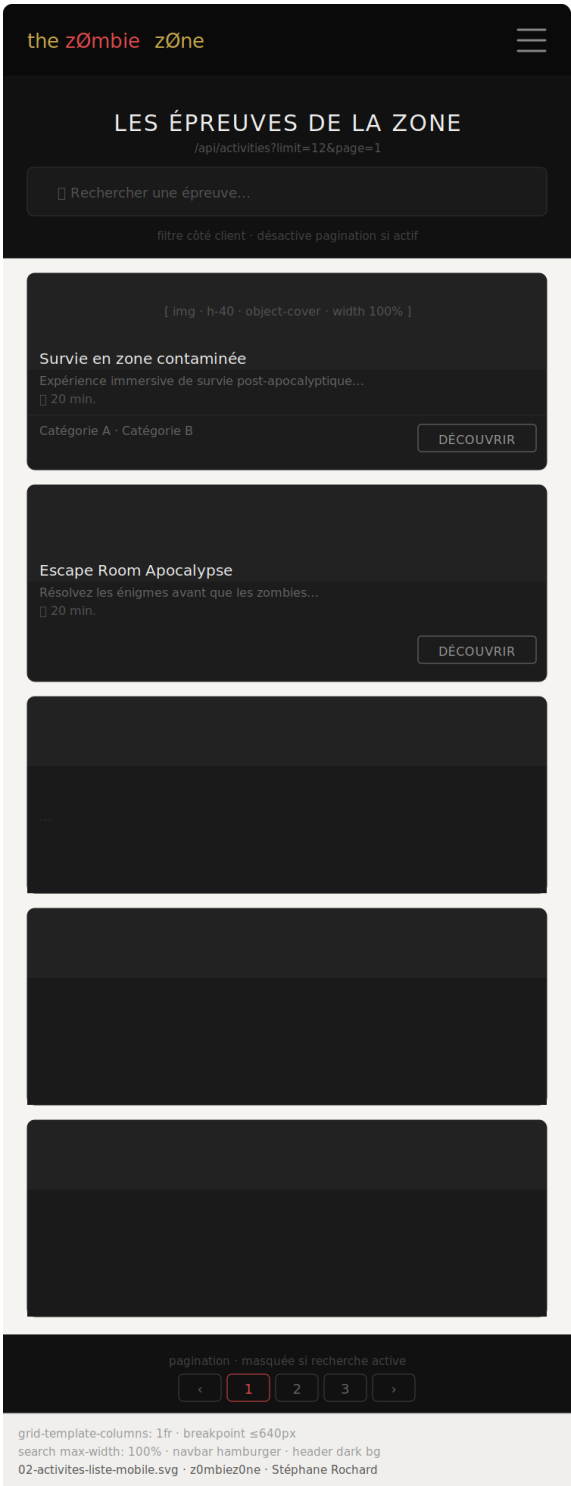
Les maquettes ont été réalisées avec Figma selon une approche mobile-first, puis déclinées en desktop. L'enchaînement ci-dessous formalise la navigation complète de l'application selon les quatre zones fonctionnelles.



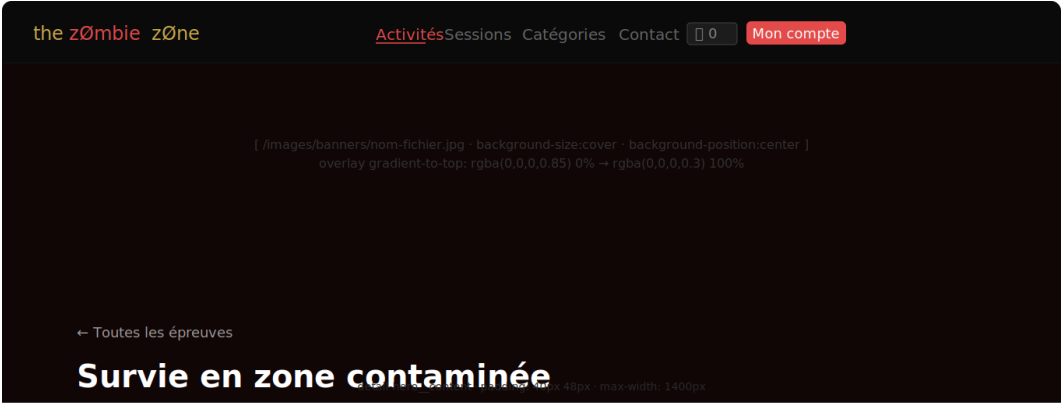


Homepage — Desktop / Mobile





Liste activités — Desktop / Mobile



↑ padding-top: 48px



□ Durée : 20 min.

SESSIONS DISPONIBLES

samedi 14 juin à 14h00 12 places	€29,90	RÉSERVER	→ /sessions/:id
dimanche 15 juin à 11h00 8 places	€29,90	RÉSERVER	
lundi 16 juin à 18h00 20 places	€29,90	RÉSERVER	

Notes techniques

route : /:slug → DynamicDetailPage (détecte type via /api/activities/by-slug/:slug)

detail-hero · height: 60vh · min-height: 400px · align-items: flex-end · bg-image cover

detail-hero__back · ← Toutes les épreuves · lien /les-epreuves · color white/70

detail-body · max-width: 900px · margin: 0 auto · padding: 48px 24px 80px

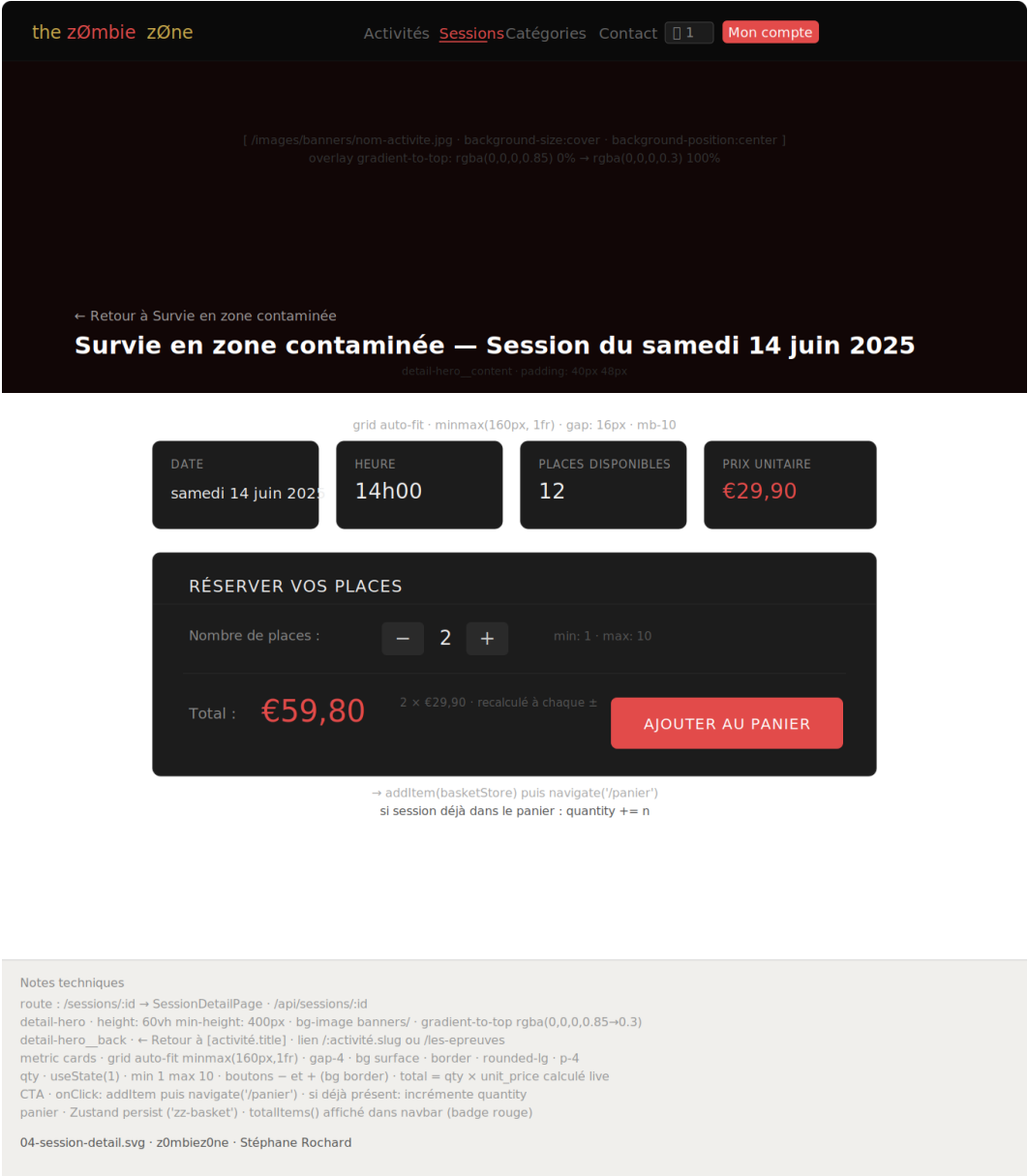
sessions · flex-col · gap-3 · chaque row : flex justify-between flex-wrap · bg surface border rounded-lg px-5 py-4

session row : date + heure (capitalize) · N places · €prix (red bold) · btn RÉSERVER → /sessions/:id

états loading : skeleton (div h-60vh + lignes) · error : "Cette épreuve n'existe pas ou a été dévorée"

03-activite-detail.svg · zØmbiezØne · Stéphane Rochard

Détail activité



Détail session

VOTRE PANIER

Survie en zone contaminée
sam. 14 juin · 14h00
€29,90 / place

–

3

+

€89,70
3 × €29,90

×

Escape Room Apocalypse
dim. 15 juin · 11h00
€39,90 / place

–

5

+

€199,50
5 × €39,90

×

Vider le panier

Total :

€289,20
€89,70 + €199,50

SE CONNECTER POUR COMMANDER

COMMANDER ← si connecté

si non connecté → navigate('/espace-client?redirectTo=/panier')
si connecté → POST /api/orders { user_id, lines: [{session_id, tickets_qty}] } → clearBasket → navigate('/espace-client')
transaction Prisma Serializable · vérif capacité restante par session avant insertion

Notes techniques

route : /panier → BasketPage · state Zustand persist ('zz-basket') · items: BasketItem[]
BasketItem : { sessionId, activityTitle, date, unitPrice, quantity }
qty controls : updateQuantity(sessionId, qty) · si qty passe à 0 → removeItem · max visuel: 10
total : totalPrice() = items.reduce(sum + unitPrice × quantity) · affiché live
commander : isAuthenticated() ? POST /api/orders : navigate /espace-client?redirectTo=/panier
navbar : totalItems() = 8 (3+5) · badge rouge sur icône

05-panier.svg · zØmbiezØne · Stéphane Rochard

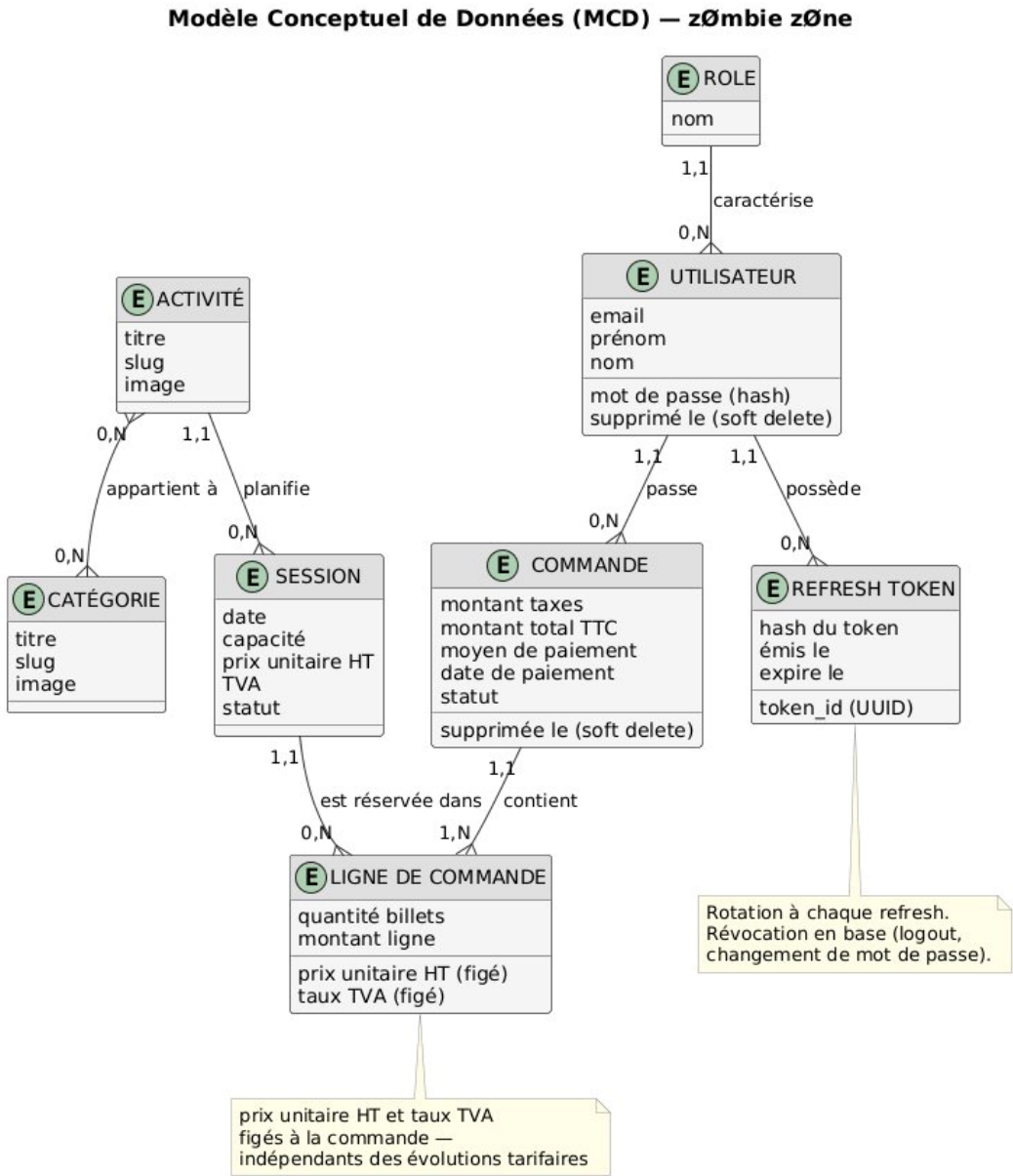
Panier / Commande

Description des écrans principaux

Écran	Description
HomePage	Vitrine immersive — hero section post-apocalyptique, mise en avant des catégories et activités phares, appels à l'action vers la réservation
ActivitiesPage	Grille d'activités avec filtre par catégorie, images WebP, prix, durée, lien vers le détail — pagination 12 éléments/page
ActivityDetailPage	Description complète, galerie (bannière + thumb), sessions disponibles triées par date, bouton 'Ajouter au panier'
BasketPage	Récapitulatif des lignes de commande, modification des quantités, calcul HT/TVA/TTC en temps réel, confirmation
DashboardPage	Historique des commandes du membre connecté avec statuts colorés, guard réseau bfcache au montage
OrderDetailPage	Détail d'une commande : lignes, sessions, dates, totaux, bouton d'annulation si éligible (statut Pending)
ManageHub (admin)	Dashboard backoffice avec navigation vers chaque CRUD — accès conditionné au rôle 'admin' (middleware requireRole)
ManageActivitiesPage	Liste paginée + filtres + actions CRUD + upload image WebP (bannière et miniature séparées)

5.3 Modèle Conceptuel de Données (MCD)

Le MCD ci-dessous présente l'ensemble des entités, leurs attributs et les cardinalités des relations. Il respecte les règles du modèle relationnel : normalisation 3NF, intégrité référentielle via les clés étrangères.



Modèle Conceptuel de Données (MCD) — notation Merise

Cardinalités et justifications

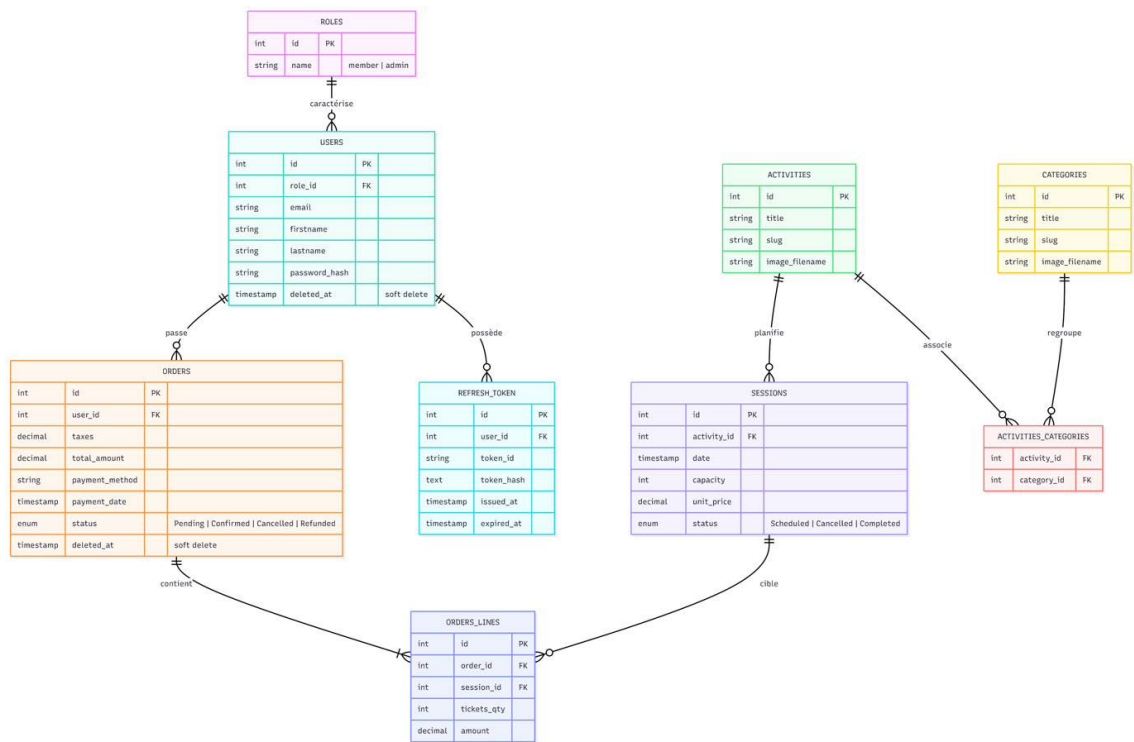
Relation	Justification
roles (1,1) — users (0,N)	Un utilisateur a exactement un rôle. Un rôle peut être attribué à plusieurs utilisateurs.
users (1,1) — RefreshToken (0,N)	Un utilisateur peut avoir plusieurs refresh tokens actifs (multi-appareils). Chaque token appartient à un seul utilisateur.
users (1,1) — orders (0,N)	Un utilisateur peut passer plusieurs commandes. Chaque commande appartient à un seul utilisateur.

activités (1,1) — activités_categories (0,N)	Table de jonction M-N : une activité peut appartenir à plusieurs catégories.
categories (1,1) — activités_categories (0,N)	Une catégorie peut regrouper plusieurs activités.
activités (1,1) — sessions (0,N)	Une activité peut avoir plusieurs sessions planifiées. Chaque session correspond à une seule activité.
sessions (1,1) — orders_lines (0,N)	Une session peut figurer dans plusieurs lignes de commande. Chaque ligne référence une session.
orders (1,1) — orders_lines (1,N)	Une commande contient au moins une ligne. Chaque ligne appartient à une commande.

Choix de conception notables

- Soft delete sur users et orders (deleted_at NULL = actif) — traçabilité RGPD et conservation pour audit
- Hard delete sur activités, categories, sessions — suppression physique intentionnelle, permet de libérer les fichiers images associés
- unit_price_ht et vat_rate figés dans orders_lines — le prix d'une activité peut changer sans affecter les commandes existantes
- token_id (UUID) dans RefreshToken — lookup O(1) par UUID, comparaison sécurisée via argon2.verify sur le hash
- slug auto-généré sur activités et categories — URLs SEO-friendly sans dépendance au nom modifiable

5.3b Modèle Physique de Données (MPD)



Modèle Physique de Données — schéma Prisma / PostgreSQL 16

5.4 Cas d'utilisation — Diagramme

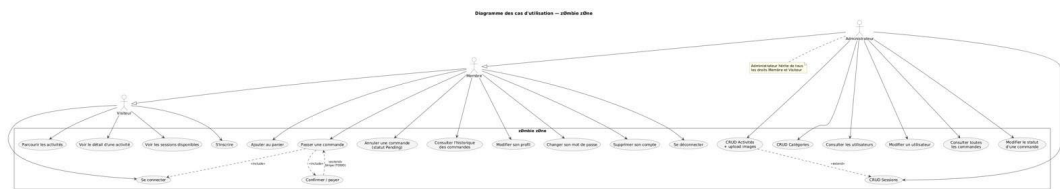


Diagramme de cas d'utilisation — Visiteur / Membre / Administrateur

Trois acteurs interagissent avec le système : Visiteur (non connecté), Membre (utilisateur authentifié), Administrateur. Chaque acteur hérite des droits de l'acteur de niveau inférieur.

VISITEUR	MEMBRE (inclut Visiteur)	ADMIN (inclut Membre)
Voir activités images	Créer/confirmer commande	CRUD activités + upload
Voir sessions	Annuler commande (Pending)	CRUD sessions
Voir catégories	Historique commandes	CRUD catégories
S'inscrire	Modifier profil/mdp	Lire/modifier utilisateurs
Se connecter	Supprimer compte	Lire/modifier commandes

5.5 Diagramme de séquence — Cycle d'authentification JWT

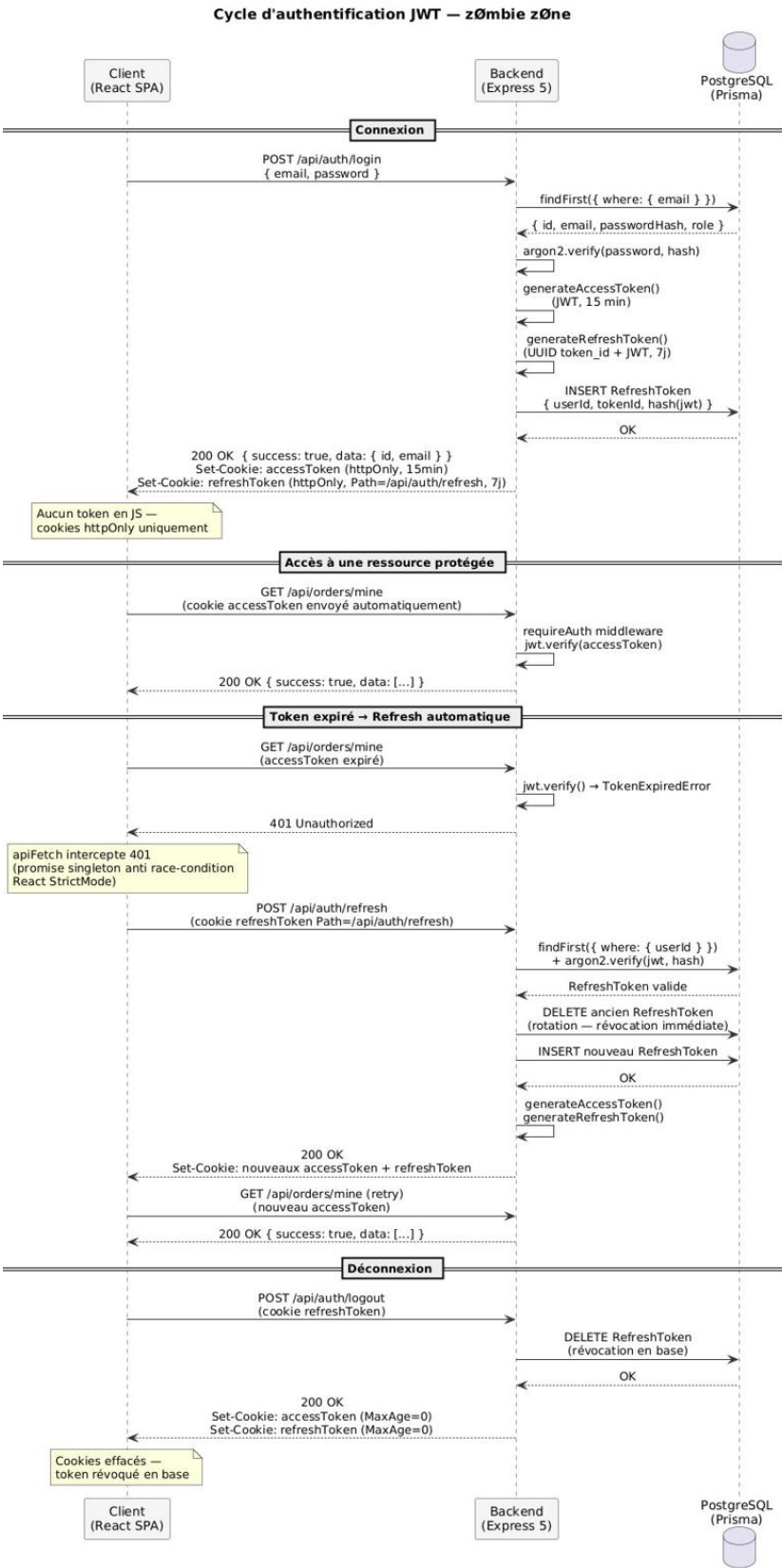
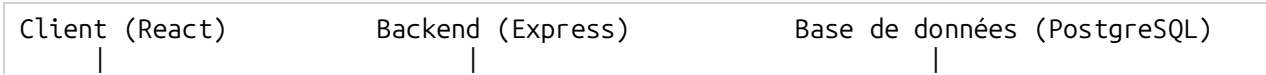


Diagramme de séquence — Cycle d'authentification JWT

Ce diagramme illustre le flux d'authentification le plus sécurisé et le plus complexe du projet : connexion, utilisation, refresh automatique et déconnexion.



```

-- POST /api/auth/login -->
{ email, password }
-- findFirst(email) -->
<-- { id, email, hash, role } --
-- argon2.verify(password)
-- generateAccessToken()
-- generateRefreshToken()
-- persistRefreshToken() -->
<-- Set-Cookie: access + refresh (httpOnly)

-- GET /api/resource -->
(cookie auto)
-- requireAuth middleware
-- jwt.verify(accessToken)
<-- 200 OK

[Token expiré]
-- GET /api/resource --> -- 401 TokenExpiredError -->
apiFetch intercepte 401
-- POST /api/auth/refresh -->
(refreshToken cookie auto) verify token_id -->
-- DELETE ancien token -->
-- INSERT nouveau token -->
<-- Set-Cookie nouveaux tokens
-- GET /api/resource (retry) -->
<-- 200 OK

```

5.6 Diagramme de classes

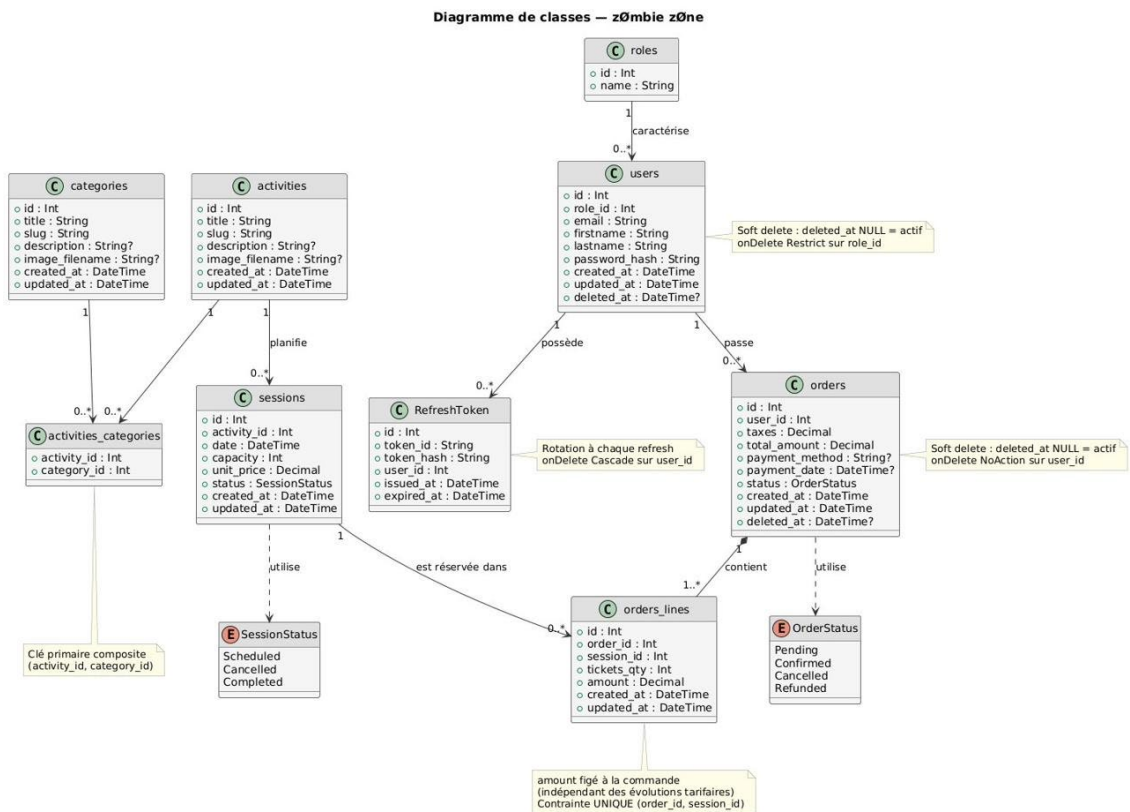


Diagramme de classes UML — modèles Prisma et enums

5.7 Cycle de vie d'une commande

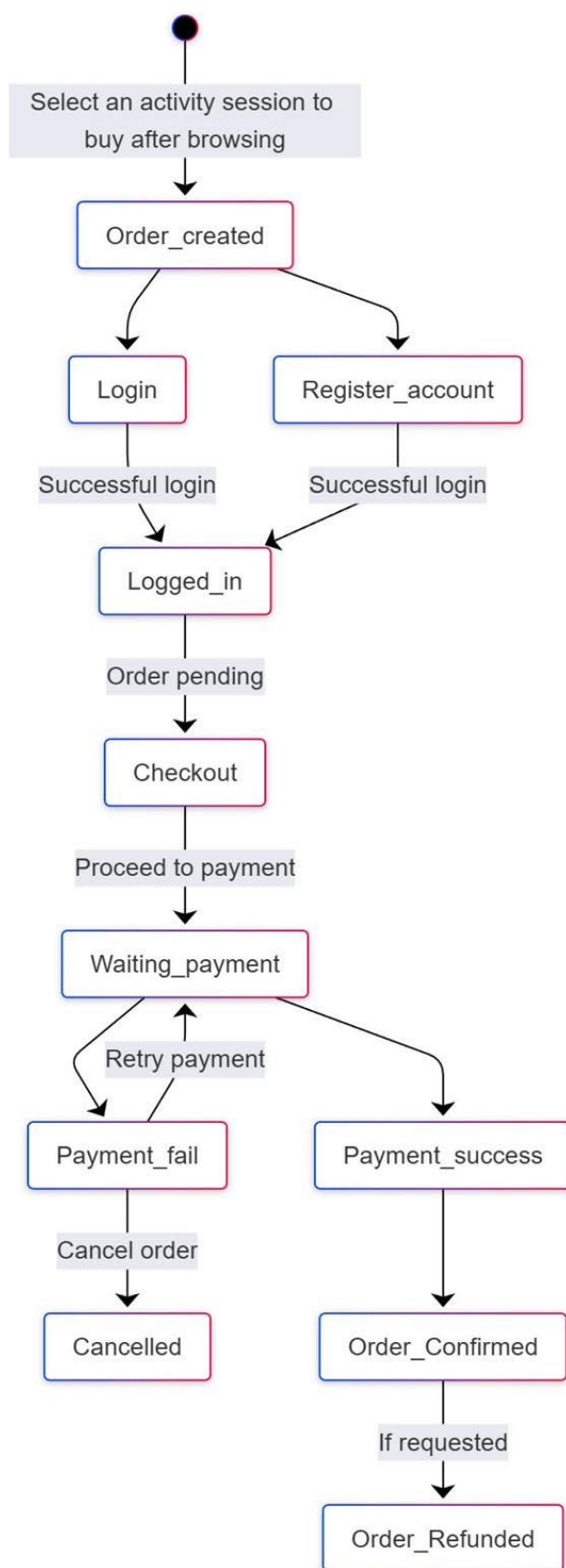


Diagramme d'états — cycle de vie d'une commande

6. Spécifications techniques et sécurité

6.1 Stack technique justifiée

Technologie	Couche	Justification
Node.js 22 + Express 5	Back-end	Runtime JS, framework HTTP léger, Express 5 avec async/await natif et meilleure gestion des erreurs
TypeScript 5 strict	Full-stack	Typage statique, aucun 'any', interfaces partagées. Détection d'erreurs à la compilation.
Prisma 5	ORM	Schéma déclaratif, migrations versionnées, client entièrement typé, select explicite anti-fuite données
PostgreSQL 16	Base de données	SGBDR ACID, contraintes FK robustes, idéal pour données relationnelles à fortes cohérences
Zod	Validation	Validation runtime sur tous les inputs body/params/query + inférence des types TypeScript
argon2id	Sécurité	Hachage mots de passe résistant GPU/ASIC — recommandé OWASP 2023, supérieur à bcrypt
jsonwebtoken	Auth	JWT access (15min) + refresh (7j), payload minimal, secrets ≥ 64 bytes aléatoires
React 19 + Vite 6	Front-end	SPA performante, HMR natif, build optimisé (tree-shaking, code splitting), concurrent features
React Router 7	Routing	Navigation SPA avec loaders, routes protégées, pas de react-router-dom (v7 unifié)
Zustand	State global	Store minimaliste sans boilerplate, persist optionnel, hydration SSR-safe si besoin futur
Biome 2.x	Qualité code	Lint + formateur unifié back + front, ~10x plus rapide qu'ESLint+Prettier
Vitest + Supertest	Tests	Runner moderne compatible TypeScript, Supertest monte l'app Express en mémoire sans port réel
Docker multi-stage	Infra	Builder (node+typescript) → Runner (node-alpine dist/ seulement). Image prod allégée.
Nginx	Reverse proxy	SSL termination, redirect HTTP→HTTPS, proxy /api/ → backend, SPA fallback (try_files)
GitHub Actions	CI/CD	Pipeline YAML déclaratif, environnement éphémère PostgreSQL de test dans le runner

6.2 Sécurité — mesures implémentées

JWT et stockage des tokens

PRINCIPE FONDAMENTAL : Les tokens JWT ne sont JAMAIS accessibles au JavaScript côté client.

```
accessToken : httpOnly; Secure; SameSite=Strict; Path=/;
MaxAge=900s (15min)
refreshToken : httpOnly; Secure; SameSite=Strict; Path=/api/auth/refresh;
MaxAge=604800s (7j)
```

- httpOnly → inaccessible à `document.cookie` et donc à toute faille XSS
- Secure → transmis uniquement sur HTTPS (jamais en HTTP clair)
- SameSite=Strict → non envoyé depuis un domaine tiers → protection CSRF native
- Path restreint → le refreshToken n'est envoyé QUE sur `/api/auth/refresh`

Référence OWASP Top 10 — couverture projet

Vulnérabilité	Statut	Mesure
A01 — Broken Access Control	☐ Traité	requireAuth + requireRole, ownership checks orders/users, soft delete, user_id depuis JWT jamais du body
A02 — Cryptographic Failures	☐ Traité	argon2id, JWT HS256, HTTPS, secrets ≥ 64 bytes, token_hash (jamais token brut en BDD)
A03 — Injection	☐ Traité	Prisma requêtes paramétrées (zéro interpolation SQL), Zod validation de tous les inputs
A05 — Misconfiguration	☐ Traité	CORS restrictif, port BDD non exposé, .env hors repo, headers sécurité Nginx
A07 — Auth Failures	☐ Traité	Cookies httpOnly, rotation refresh, expiration courte, déduplication refreshes concurrents
A09 — Logging	☐ Partiel	Logs Winston configurés avec niveaux par env, monitoring en production à renforcer

6.3 Traçabilité d'une fonctionnalité dans le code (CP6)

Exemple : création d'une commande — traçabilité complète depuis la route jusqu'à la base de données.

1. Route
backend/src/routers/orders.router.ts
POST /api/orders → [requireAuth] → createOrder
2. Middleware
requireAuth vérifie le cookie accessToken (JWT)
→ payload { id, role } attaché à req.user
3. Validation
Zod schema : `body` vide acceptable (user_id vient de req.user)
createOrderSchema.parse(req.body)
4. Controller
backend/src/controllers/orders.controller.ts

```
const userId = req.user!.id; // JAMAIS req.body.user_id
const order = await prisma.$transaction(async (tx) => {
  return tx.orders.create({
    data: { user_id: userId, status: 'Pending' },
    select: { id: true, status: true, created_at: true }
  });
});
return res.status(201).json({ success: true, data: order });
```
5. BDD
INSERT INTO orders (user_id, status, created_at) VALUES (?, 'Pending', NOW())
→ Retour : { id, status: 'Pending', created_at }

7. Réalisations — Extraits de code significatifs

7.1 Authentification JWT — Backend

Controller loginUser

```
// backend/src/controllers/auth.controller.ts
export const loginUser = async (req: Request, res: Response) => {
  const { email, password } = loginSchema.parse(req.body); // Zod
  const user = await prisma.users.findFirst({
    where: { email, deleted_at: null },
    select: { id: true, email: true, password_hash: true, role: { select: { name: true } } }
  });
  if (!user) throw new UnauthorizedError('Invalid credentials');
  const valid = await argon2.verify(user.password_hash, password);
  if (!valid) throw new UnauthorizedError('Invalid credentials');
  const { accessToken } = generateAccessToken({ id: user.id, role: user.role.name });
  const { jwt: refreshJwt, tokenId } = generateRefreshToken(user.id);
  await persistRefreshToken(user.id, tokenId, refreshJwt); // hash argon2 en base
  setAccessToken(res, accessToken); // httpOnly + Secure + SameSite=Strict
  setRefreshCookie(res, refreshJwt);
  return res.json({ success: true, data: { id: user.id, email: user.email } });
};
```

apiFetch — Intercepteur 401 avec déduplication

```
// vite-frontend/src/lib/apiFetch.ts
let refreshPromise: Promise<void> | null = null; // Déduplication React StrictMode

export const apiFetch = async (url: string, options: RequestInit = {}):
Promise<Response> => {
  const res = await fetch(url, { ...options, credentials: 'include' });
  if (res.status !== 401) return res;
  // Un seul refresh même si plusieurs requêtes concurrentes déclenchent un 401
  if (!refreshPromise) {
    refreshPromise = useAuthStore.getState().refreshToken()
      .finally(() => { refreshPromise = null; });
  }
  await refreshPromise;
  return fetch(url, { ...options, credentials: 'include' }); // retry
};
```

7.2 Composants d'accès aux données — Prisma (CP8)

Pagination — helper réutilisable

```
// backend/src/helpers/pagination.ts
export const getPagination = (query: { page?: string; limit?: string }) => {
  const page = Math.max(1, parseInt(query.page ?? '1', 10));
  const limit = Math.min(50, Math.max(1, parseInt(query.limit ?? '12', 10)));
  return { skip: (page - 1) * limit, take: limit, page, limit };
};
// Utilisé dans tous les GET de liste : activities, sessions, orders, users
// Max 50 éléments/page pour prévenir les requêtes abusives (éco-conception)
```

Transaction Prisma — createOrderLine avec vérification capacité

```
// backend/src/controllers/orders_lines.controller.ts
export const createOrderLine = async (req: Request, res: Response) => {
  const { order_id, session_id, quantity } =
    createOrderLineSchema.parse(req.body);
  const line = await prisma.$transaction(async (tx) => {
    // Vérification capacité disponible dans la transaction (évite les conflits concurrent)
    const session = await tx.sessions.findUniqueOrThrow({
      where: { id: session_id },
      select: { capacity: true, status: true, activity: { select: { price: true } }
    }
  });
  if (session.status !== 'Scheduled')
    throw new BadRequestError('Session not available');
  const booked = await tx.orders_lines.aggregate({
    where: { session_id },
    _sum: { quantity: true }
  });
  const available = session.capacity - (booked._sum.quantity ?? 0);
  if (quantity > available)
    throw new BadRequestError(`Only ${available} seats available`);
  return tx.orders_lines.create({
    data: { order_id, session_id, quantity,
      unit_price_ht: session.activity.price, // prix figé au moment de la commande
      vat_rate: 0.20 },
    select: { id: true, quantity: true, unit_price_ht: true, vat_rate: true }
  });
};
return res.status(201).json({ success: true, data: line });
};
```

7.3 Interface utilisateur — React

Guard bfcache — DashboardPage

```
// vite-frontend/src/pages/dashboard/DashboardPage.tsx
const DashboardPage = () => {
  const { user, refreshToken, isInitialized } = useAuthStore();
  const navigate = useNavigate();

  // Guard réseau au montage – protège contre le bfcache (bouton 'Précédent')
  // Le browser peut restaurer la page depuis la mémoire sans re-exécuter le JS
  useEffect(() => {
    if (!isInitialized) return;
    const checkSession = async () => {
      try {
        await refreshToken(); // vérification réseau effective, pas seulement l'état Zustand
      } catch {
        navigate('/login', { replace: true });
      }
    };
    checkSession();
  }, [isInitialized]);
  // ...
};
```

8. Éléments de sécurité de l'application

8.1 Veille sécurité effectuée

Sujet de veille	Résultat et impact sur le code
JWT httpOnly cookies	Recherche : 'JWT httpOnly cookie XSS CSRF SPA 2024'. Sources : OWASP JWT Cheat Sheet, RFC 6265. Résultat : abandon de l'approche localStorage initialement envisagée, adoption des cookies httpOnly+SameSite=Strict.
Refresh token rotation	Recherche : 'refresh token rotation concurrent requests race condition'. Sources : Auth0 docs, hasura.io. Découverte du problème React StrictMode (double-invoke). Solution : promise singleton refreshPromise.
argon2 vs bcrypt	Recherche : 'argon2 bcrypt password hashing OWASP 2023'. Sources : OWASP Password Storage Cheat Sheet. Choix argon2id confirmé — résistant GPU/ASIC, recommandé OWASP depuis 2023.
Soft delete RGPD	Recherche : 'soft delete GDPR right erasure implementation'. Source : CNIL guide développeurs. deleted_at sur users/orders : données masquées mais conservées pour audit. Suppression définitive possible ultérieurement.
Injection SQL	Recherche : 'Prisma SQL injection prevention parameterized queries'. Source : documentation Prisma. Confirmation : Prisma utilise des requêtes paramétrées — aucune interpolation de chaîne possible via l'ORM.

8.2 Failles identifiées et corrections

Vulnérabilité	Référence	Correction appliquée
user_id dans req.body	BUG-B1	La création de commande utilisait req.body.user_id — un utilisateur pouvait créer une commande au nom d'un autre. Correction : user_id exclusivement depuis req.user.id (payload JWT).
deleteUser hard delete	BUG-B2	La suppression d'utilisateur était physique (DELETE FROM users). Correction : soft delete avec deleted_at = new Date(). Conservation des commandes liées.
updateUser sans Zod	BUG-B3	Le controller de mise à jour utilisateur n'avait pas de validation Zod. Correction : ajout du schéma updateUserSchema avec validation stricte.
Zustand persist token	Anomalie #5	Le basketStore utilisait persist() avec localStorage. Correction : suppression du persist, localStorage.removeItem() explicite à la déconnexion.

9. Plan de tests

9.1 Stratégie de tests

La stratégie suit une pyramide priorisée par ROI. Vitest (runner) + Supertest (simulation HTTP) : Supertest monte l'app Express directement en mémoire (`http.createServer`) sans port réseau — tests rapides et sans effet de bord.

Type	Priorité	Statut et couverture
Intégration	Priorité 1	116 tests — route HTTP → controller → BDD (plusieurs couches). Meilleur ROI pour un backend Express.
Unitaires	Priorité 2	35 tests — fonctions isolées (helpers, utils, lib). Colocalisés avec le fichier testé.
E2E Playwright	Priorité 3	Prévu — flow navigateur complet. Non implémenté dans le MVP.

9.2 Tests d'intégration — 116 tests

Fichier	Tests	Scénarios couverts
auth.test.ts	18	register, login, logout, refresh (rotation + révocation), profile, cookies httpOnly, 401 token expiré vs invalide
activities.test.ts	19	CRUD complet, slug auto-généré, hard delete, pagination, 401 non auth, 403 non admin, 404 not found
categories.test.ts	22	CRUD, slug auto, hard delete, 409 Conflict si liée à une activité, validation Zod 400
sessions.test.ts	24	CRUD, filtre par statut, hard delete, 400 Bad Request si order_lines existantes, validation capacité
orders.test.ts	21	POST/GET/PUT/DELETE, calcul HT/TTC, user_id depuis JWT, transitions statut, soft delete
users.test.ts	12	GET liste (admin only), GET/:id (owner ou admin), PUT profil, PUT password révocation refresh tokens, soft delete

9.3 Tests unitaires — 35 tests

Fichier	Tests	Couverture
tokens.test.ts	12	generateAccessToken/generateRefreshToken — payload, expiration, signature, token_id UUID
auth.test.ts	8	Schémas Zod login/register — email invalide, politique mdp (min 8, majuscule, chiffre, spécial)
slugify.test.ts	8	Cas nominaux, accents, caractères spéciaux, espaces multiples, chaîne vide, unicité
getPagination.test.ts	7	Defaults page=1/limit=12, bornes min/max (max 50), calcul offset, types de retour TypeScript

9.4 Infrastructure de tests

Élément	Description
BDD de test isolée	zombiezone_test (TEST_DATABASE_URL dans .env) — jamais la BDD de dev ou de production
Reset entre chaque suite	resetDatabase() dans beforeEach — vide toutes les tables dans l'ordre FK, recrée les rôles via skipDuplicates
Isolation totale	Pas de dépendance entre tests — chaque test part d'une BDD vierge et crée ses propres données
Cookies dans les tests	request.agent(app) de Supertest pour les tests nécessitant des cookies persistants entre requêtes
CI GitHub Actions	PostgreSQL de test éphémère dans le runner — tests lancés à chaque push et PR sur master

10. Jeu d'essai — Fonctionnalité la plus représentative

Fonctionnalité choisie : cycle complet d'authentification JWT. C'est la fonctionnalité la plus significative du projet du point de vue sécurité et architecture.

10.1 Scénarios testés

Scénario 1 — Login avec identifiants valides

Paramètre	Valeur
Endpoint	POST /api/auth/login
Données en entrée	{ "email": "user@zombiezone.fr", "password": "zØmbie zØne@2024" }
Données attendues	200 OK — Set-Cookie: accessToken (httpOnly) + Set-Cookie: refreshToken (httpOnly, Path=/api/auth/refresh)
Données obtenues	☐ 200 OK — body: { success: true, data: { id, email } } — deux cookies httpOnly définis
Écart	Aucun — conforme aux spécifications

Scénario 2 — Login avec mauvais mot de passe

Paramètre	Valeur
Données en entrée	{ "email": "user@zombiezone.fr", "password": "WrongPassword" }
Données attendues	401 Unauthorized — message générique identique à 'email inconnu' (anti-oracle)
Données obtenues	☐ 401 — { success: false, error: 'Invalid credentials' } — même message dans les deux cas
Écart	Aucun — protection contre l'énumération d'emails confirmée

Scénario 3 — Token expiré → Refresh automatique

Paramètre	Valeur
Setup	Token accessToken expiré simulé — refreshToken valide en base
Séquence	GET /api/auth/profile → 401 → apiFetch intercepte → POST /api/auth/refresh → GET /api/auth/profile (retry)
Données attendues	200 OK final — nouveau accessToken en cookie, ancien refreshToken supprimé en base (rotation)
Données obtenues	☐ 200 OK — rotation vérifiée : ancien token_id absent de RefreshToken, nouveau présent
Écart	Aucun — intercepteur apiFetch fonctionne de manière transparente

Scénario 4 — Accès à une ressource sans authentification

Paramètre	Valeur
Endpoint	GET /api/orders/mine (sans cookie)
Données attendues	401 Unauthorized — { success: false, error: 'No token' }
Données obtenues	□ 401 — guard clause du middleware requireAuth déclenché avant le controller
Écart	Aucun

Scénario 5 — Logout + tentative d'utilisation du refreshToken révoqué

Paramètre	Valeur
Étape 1	POST /api/auth/logout → 200 OK, cookies effacés, RefreshToken supprimé en base
Étape 2	POST /api/auth/refresh avec l'ancien refreshToken → attendu : 401
Données obtenues	□ 401 — token_id non trouvé en base (révocation effective)
Écart	Aucun — la révocation fonctionne correctement

10.2 Analyse des écarts

Aucun écart constaté entre les données attendues et obtenues sur ce jeu d'essai. Les 18 tests automatisés du fichier auth.test.ts reproduisent ces scénarios de manière exhaustive et reproductible.

Point d'amélioration identifié lors de ce jeu d'essai : absence de rate limiting sur POST /api/auth/login (protection anti-brute-force). Documenté dans le backlog comme amélioration post-MVP.

11. Déploiement — Procédure et documentation

11.1 Infrastructure de production

Paramètre	Configuration
Hébergeur	VPS Ionos — Ubuntu 24.04 LTS — 2 vCPU / 2 GB RAM / 80 GB NVMe
Domaines	sharo.fr — DNS A 82.165.180.54
SSL	Let's Encrypt (Certbot) — renouvellement automatique — cookies Secure flag actif
Nginx	Reverse proxy : HTTP→HTTPS redirect, proxy /api/ → backend:3000, SPA fallback try_files
PostgreSQL	Port 5432 interne Docker uniquement — non exposé sur le réseau public

Diagramme de déploiement — zombie zone

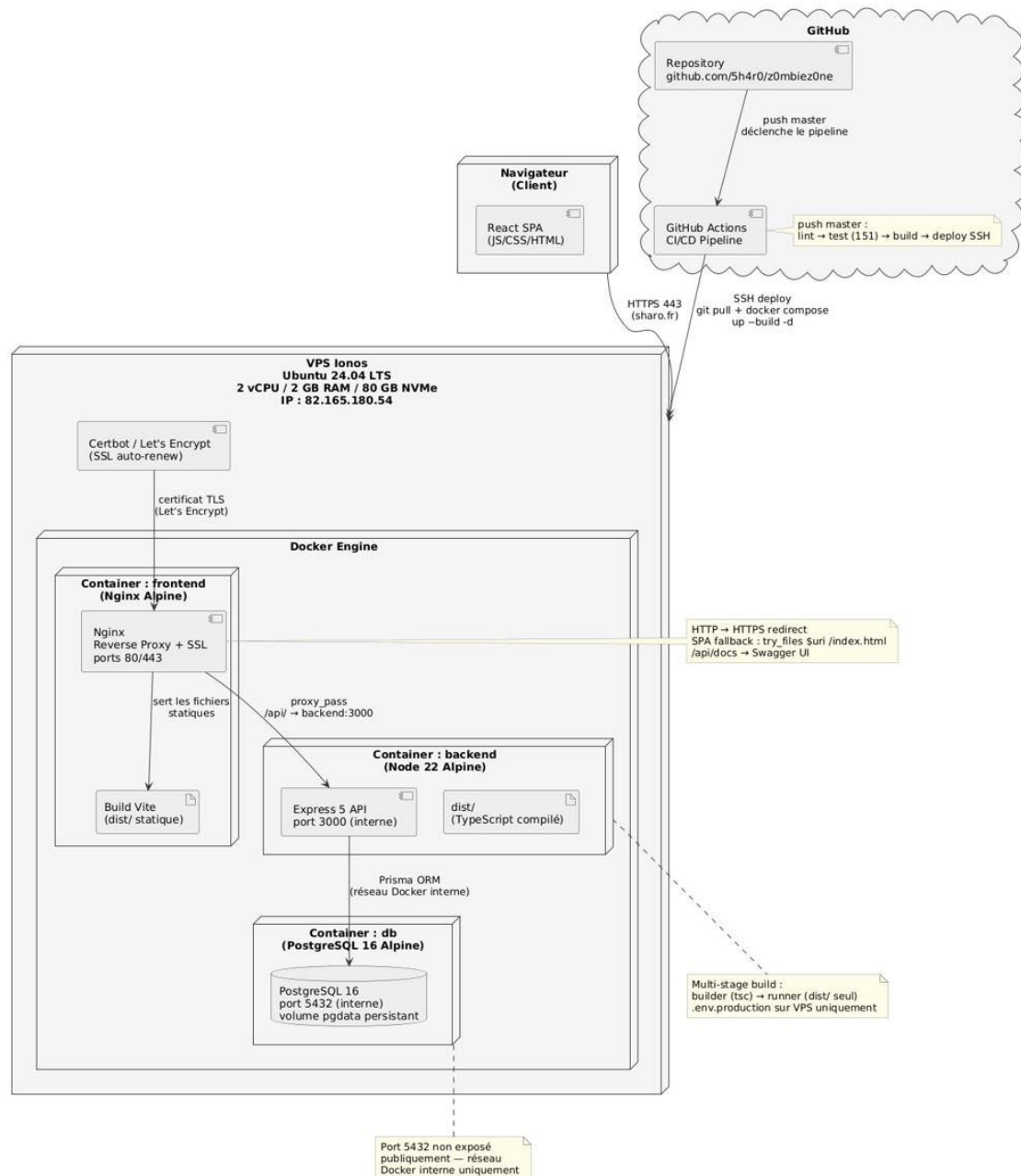


Diagramme de déploiement — VPS Ionos / Docker / GitHub Actions

11.2 Architecture Docker (multi-stage)

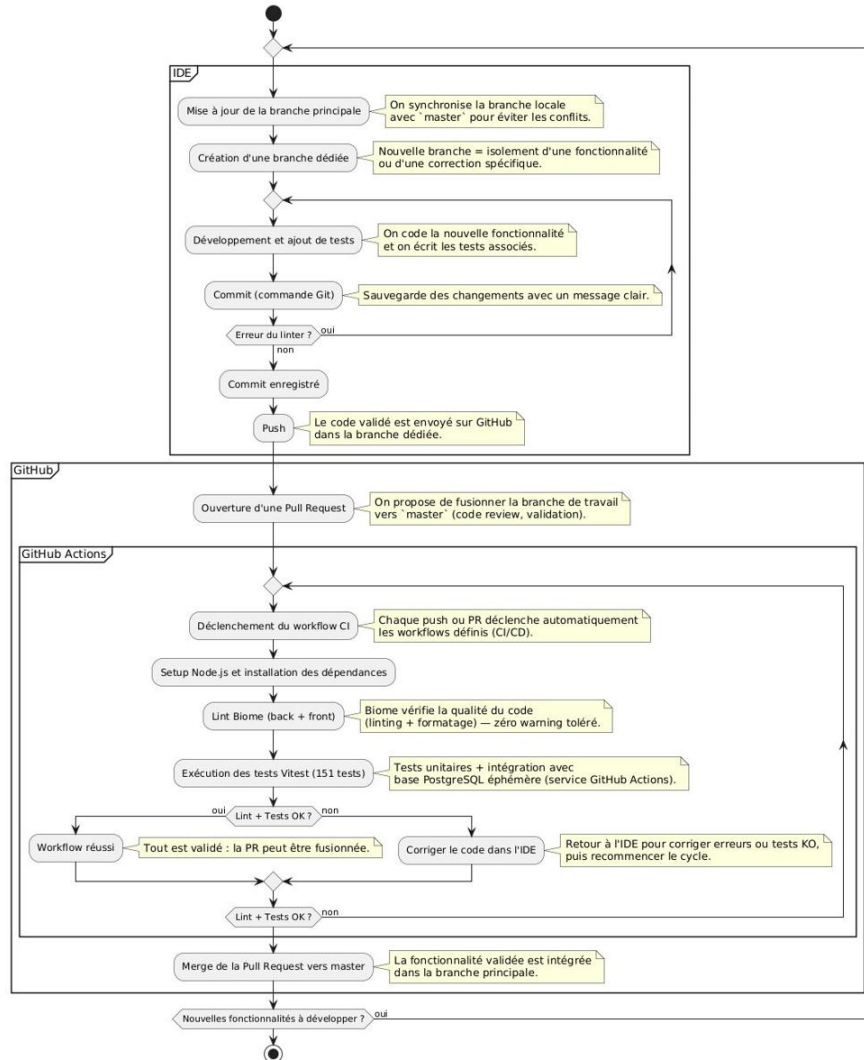
```
# docker/Dockerfile.backend - Multi-stage build
FROM node:22-alpine AS builder
COPY . .
RUN npm ci && npx prisma generate && npm run build # tsc → dist/

FROM node:22-alpine AS runner
COPY --from=builder /app/dist ./dist # uniquement le code compilé
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/backend/src/models ./backend/src/models # migrations
RUN ln -s ./backend/src/models/migrations ./backend/prisma/migrations # symlink
Prisma
CMD ['node', 'dist/index.js']

Services Docker Compose prod :
```

db → PostgreSQL 16 Alpine (volume pgdata persistant)
 backend → Node 22 Alpine (dist/, env.production)
 frontend → Nginx Alpine (build Vite statique + nginx.conf)

11.3 Pipeline CI/CD — GitHub Actions



Pipeline CI/CD — GitHub Actions (lint → tests → build → deploy)

```

# .github/workflows/ci.yml - déclenché sur push master + toutes PR

jobs:
  ci:
    services:
      postgres: { image: postgres:16, env: { POSTGRES_DB: zombiezone_test } }
    steps:
      - Checkout + setup Node 22
      - npm run install:all
      - cd backend && npm run lint ← Biome check
      - cd vite-frontend && npm run lint
      - cd backend && npm run test ← 151 tests Vitest
      - npm run build ← tsc + vite build
      # Déploiement uniquement sur push master :
      - if: github.ref == 'refs/heads/master'
        run: ssh deploy@$VPS git pull && docker compose up --build -d
  
```

11.4 Procédure de mise à jour

1. git push origin master — déclenche le pipeline CI/CD
2. GitHub Actions : lint ☐ → tests ☐ → build ☐ → deploy SSH
3. VPS : git pull + docker compose -f docker-compose.prod.yml up -d --build
4. Si nouvelles migrations : npx prisma migrate deploy dans le container backend
5. Vérification : curl https://sharo.fr/api/activities → 200 OK

12. Difficultés rencontrées et améliorations

12.1 Difficultés techniques

Race conditions sur le refresh token (React StrictMode)

React 18+ en mode strict double-invoque les effets au montage. Deux appels simultanés à `/api/auth/refresh` consommaient le même token — le second appel échouait car le token était déjà rotaté.

Analyse : logs en mode développement + lecture documentation React StrictMode + posts Auth0/hasura sur le sujet.

Solution : singleton de promesse (let refreshPromise: Promise<void> | null = null) dans apiFetch et dans le store Zustand pour dédupliquer les appels concurrents.

bfcache — restauration de page sans re-exécution JavaScript

Le bouton 'Précédent' peut restaurer une page protégée depuis la mémoire du navigateur sans re-exécuter le JavaScript — un utilisateur déconnecté pouvait voir son espace client.

Solution : Cache-Control: no-store sur les réponses des routes protégées + vérification réseau au montage de chaque page d'espace client (guard réseau, pas uniquement l'état Zustand en mémoire).

Docker multi-stage et migrations Prisma

Prisma cherche les migrations dans `prisma/migrations` relatif au CWD (`/app/backend`), mais notre schéma est dans `backend/src/models/migrations/`. En production, la commande `migrate deploy` échouait avec 'No migration found'.

Solution : création d'un symlink `backend/prisma/migrations` → `backend/src/models/migrations` dans le Dockerfile, documentée dans `DEPLOY.md` avec les autres pièges connus.

Biome 2.x en monorepo

L'exécution de Biome depuis la racine déclenchait un conflit de configuration (`biome.json` racine + `biome.json` dans `vite-frontend/`).

Solution : exécution de Biome depuis chaque workspace (`cd vite-frontend && npx biome check src/`) — documentée dans `CLAUDE.md` pour ne pas être re-découverte.

12.2 Améliorations futures

Amélioration	Description et priorité
Rate limiting	<code>express-rate-limit</code> sur <code>/api/auth/*</code> — protection anti-brute-force. Actuellement absent du MVP.
Emails transactionnels	Nodemailer/Resend : confirmation inscription + confirmation commande avec détail lignes HT + total TTC.
Paieement Stripe	Intégration Stripe pour la confirmation de commande — prévu en V2 (hors MVP selon le cahier des charges).
Tests E2E Playwright	Flow complet navigateur : login → réservation → confirmation → annulation.

i18n FR/EN	Support multilingue prévu dans l'architecture (routes /fr et /en), non implémenté.
RGAA complet	Audit accessibilité pour dépasser 60% — tests avec lecteur d'écran, navigation clavier complète.

12.3 Conclusion

zØmbie zØne est une application web fullstack TypeScript complète, déployée en production sur <https://sharo.fr>. Le projet couvre l'ensemble des 11 compétences du référentiel CDA, avec une attention particulière portée à la sécurité (JWT httpOnly, argon2id, Zod, OWASP), à la qualité du code (151 tests automatisés, CI/CD, Biome, TypeScript strict) et à la documentation exhaustive (DEPLOY.md, TESTS.md, Swagger, CLAUDE.md).

Ce projet m'a permis de mettre en pratique des choix techniques professionnels motivés, de gérer les problématiques d'infrastructure réelles (Docker, VPS, SSL, CI/CD) et de développer une sensibilité concrète aux enjeux de sécurité applicative. Je suis en mesure d'expliquer, de justifier et de défendre l'ensemble des décisions prises tout au long du développement.

GitHub

:

<https://github.com/5h4r0/z0mbiez0ne>

Production

:

<https://sharo.fr>

API docs

:

<https://sharo.fr/api/docs>

Tests

:

151 automatisés (116 intégration + 35 unitaires)

Stack

:

[Express 5](#) • [Prisma](#) • [PostgreSQL](#) • [React 19](#) • [Vite](#) • [Zustand](#) • [Docker](#)

:

[GitHub Actions](#)

Annexes

A. Endpoints API complets

Endpoint	Auth	Description
POST /api/auth/register	—	Créer un compte
POST /api/auth/login	—	Se connecter
POST /api/auth/refresh	—	Rafraîchir le token
POST /api/auth/logout	—	Se déconnecter
GET /api/auth/profile	member, admin	Profil courant
GET /api/activities	—	Liste (paginée)
GET /api/activities/by-slug/:slug	—	Détail par slug
GET /api/activities/:id	—	Détail par id
POST /api/activities	admin	Créer
PUT /api/activities/:id	admin	Modifier
DELETE /api/activities/:id	admin	Supprimer
GET /api/categories	—	Liste
GET /api/sessions	—	Liste
POST /api/sessions	admin	Créer
PUT /api/sessions/:id	admin	Modifier
DELETE /api/sessions/:id	admin	Supprimer
GET /api/orders	admin	Toutes les commandes
GET /api/orders/mine	member, admin	Mes commandes
POST /api/orders	member, admin	Créer commande
PUT /api/orders/:id	member, admin	Modifier statut
DELETE /api/orders/:id	member, admin	Supprimer (Pending)
GET /api/users	admin	Liste utilisateurs
GET /api/users/:id	member, admin	Détail utilisateur
PUT /api/users/:id	member, admin	Modifier profil
PUT /api/users/:id/password	member, admin	Changer mot de passe
DELETE /api/users/:id	member, admin	Supprimer compte
POST /api/upload/activity-banner	admin	Upload bannière WebP
POST /api/upload/activity-thumb	admin	Upload miniature WebP

B. Variables d'environnement

```
# backend/.env (dev) / backend/.env.production (prod – sur VPS uniquement, jamais
committé)
```

```
DATABASE_URL=postgresql://user:password@localhost:5432/zombiezone
```

```
PORT=3000
NODE_ENV=development|production
ALLOWED_ORIGINS=http://localhost:5173|https://sharo.fr
JWT_ACCESS_SECRET=<64+ bytes généré via crypto.randomBytes(64).toString('hex')>
JWT_REFRESH_SECRET=<64+ bytes différent du précédent>
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
ADMIN_EMAIL=admin@zombiezone.fr
ADMIN_FIRSTNAME=Admin
ADMIN_LASTNAME=zombie zone
ADMIN_PASSWORD=<mot de passe fort – sans '$' pour éviter l'interprétation shell>
LOG_LEVEL=info|warn

# Racine VPS – .env (jamais dans le repo)
POSTGRES_USER=zombiezone
POSTGRES_PASSWORD=<même password que DATABASE_URL>
```